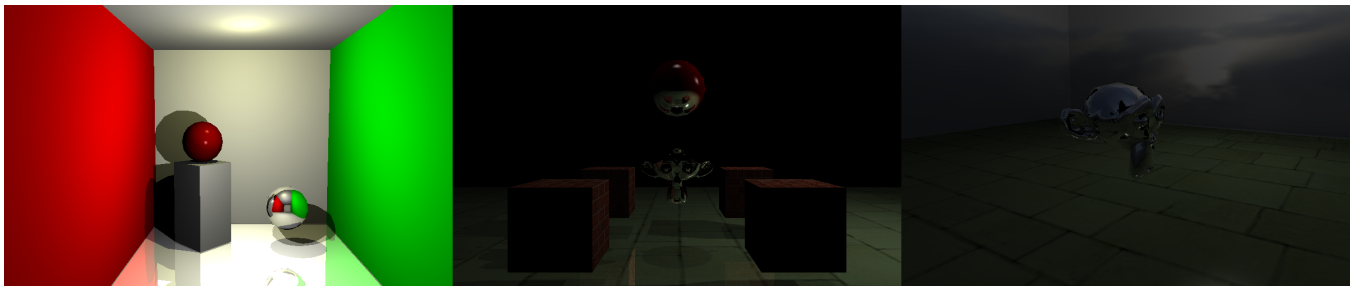
Uma Arquitetura Híbrida otimizando Ray Tracing para Aplicações em Tempo Real

Diego Luís Nogueira Gonçalves
Rossana Baptista Queiroz
Vinícius Cassol

Universidade do Vale do Rio dos Sinos



Resumo

Este artigo descreve uma arquitetura híbrida que implementa a técnica de renderização de imagens conhecida como *ray tracing*. Esta arquitetura foi desenvolvida como uma otimização da técnica, tendo em vista possibilitar seu uso em aplicações em tempo real, como jogos digitais.

Devido à complexidade do *ray tracing*, é usual a utilização de técnicas de otimização para melhorar seu desempenho, como estruturas de dados espaciais. A arquitetura descrita neste artigo utiliza como forma de otimização da técnica de *ray tracing* uma combinação desta com a técnica de *deferred shading*.

Keywords: Ray Tracing, Real-time Rendering, GPU Computing, Deferred Shading

Author's Contact:

dlngoncalves@gmail.com

1 Introdução

A busca por imagens com maior grau de realismo sempre norteou os desenvolvimentos da computação gráfica. Esta busca se estende também aos jogos digitais. A técnica conhecida como *ray tracing* permite um alto grau de realismo na geração de imagens. A técnica possibilita simular fenômenos físicos como reflexões, refrações, e sombreamento.

Entretanto, devido ao seu alto custo computacional, *ray tracing* encontrou pouco uso em aplicações em tempo real, como jogos digitais. Uma particularidade da técnica de *ray tracing* é que ela é facilmente paralelizável [García et al. 2012]. Ao longo da última década, o principal desenvolvimento na arquitetura de processadores gráficos foi o aumento de sua capacidade de processamento paralelo. De fato, GPUs possuem uma arquitetura que mais se assemelha a processadores vetoriais de supercomputadores do que aquela empregada por CPUs tradicionais [Cook 2013]. Houve assim um aumento no número de pesquisas procurando implementar ray tracing em GPUs, tendo em vista o possível ganho de performance obtido com a paralelização do algoritmo. Em geral a técnica é implementada em GPU em sua integralidade, ou de uma forma híbrida, combinada com outros métodos de renderização.Tendo em vista estes desenvolvimentos, o presente trabalho tem como objetivo a implementação de uma de uma arquitetura híbrida, combinando aspectos das técnicas de *ray tracing* e *deferred rendering*.A combinação destas técnicas permite explorar de maneira vantajosa as particularidades de GPUs modernas, como uso do algoritmo de *Z-buffer* implementado em hardware, para a determinação da visibilidade das superfícies de maneira mais eficiente que em um *ray tracer* tradicional, sem perda de qualidade visual. Também foi realizada uma análise do desempenho da arquitetura, em comparação com um *ray tracer* tradicional.

[Cabeleira 2010], propõe uma *engine* gráfica que combina rasterização e técnicas de *ray tracing* para aproximar iluminação global em tempo real, obtendo resultados satisfatórios. Em [García et al. 2012] bons resultados foram obtidos ao se utilizar o estágio do pipeline gráfico destinado geralmente ao processamento não-gráfico (o *compute shader*). Por fim, [Wang et al. 2014] demonstra a possível aplicação de ray tracing em GPUs móveis.

2 Trabalhos Relacionados

Inúmeras pesquisas tem sido feitas acerca do tópico de *ray tracing* em GPUs visando seu uso em aplicações em tempo real. Um desenvolvimento importante no âmbito de *ray tracing* em GPUs foi o lançamento da API OptiX, desenvolvida pela Nvidia, para auxiliar no desenvolvimento de aplicações que utilizem *ray tracing* em GPU. [Parker et al. 2010]

[Bikker and van Schijndel 2013] Apresenta a *Brigade Engine*, uma *engine* gráfica que utiliza *path tracing* em GPU em tempo real. A técnica de *path tracing* é relacionada ao *ray tracing*, obtendo uma qualidade visual superior, porém com maior custo computacional. A *Brigade Engine* consegue obter boa qualidade gráfica em tempo real, entretanto artefatos visuais são aparentes à medida que novas imagens são geradas. Isto ocorre devido à características da técnica de *path tracing*, cuja descrição detalhada foge do escopo do presente trabalho.

[Movania 2013] Descreve a implementação de um *Ray Tracer* em GPU, onde toda a computação é feita no *fragment shader*. Nesta implementação, os modelos 3D da cena a ser renderizada são armazenados em texturas, com as posições dos vértices e outros atributos sendo representadas pelos componentes rgb da textura. Estas texturas são passadas ao *fragment shader*, onde os dados são extraídos e utilizados para os cálculos necessários para o *ray tracing*.

Um *Ray Tracer* híbrido, com raios primários acelerados através do uso do pipeline gráfico tradicional e do uso do algoritmo de *depth buffer* é descrito em [Sabino et al. 2011]. Esta arquitetura hibrida permite que apenas alguns objetos da cena sejam afetados pelo *ray tracer* e os demais sejam renderizados pelo pipeline gráfico tradicional.

Outra arquitetura híbrida é apresentada em [Pedrosa 2012],

implementado um modelo de renderização híbrida utilizando ray tracing em tempo real, com heurísticas visando a melhora do desempenho. A implementação híbrida envolve o uso da técnica de *deferred shading*. Além disso, a heurística utilizada garante que apenas elementos da cena tridimensional a ser renderizada que se beneficiem do *ray tracing* sejam renderizados desta forma. Também é considerado um tempo máximo para a geração das imagens, e caso este seja ultrapassado, a cena é renderizada de maneira tradicional. A tendencia do sistema visual humano de não perceber igualmente detalhes em todos os pontos de uma imagem, focando apenas em elementos principais, é explorada nesta arquitetura. Neste caso, não são apenas objetos da cena que são renderizados utilizando *ray tracing*, mas a região do espaço ao redor destes objetos principais, a fim de minimizar discontinuiades muito aparentes.

3 Arquitetura do Sistema

A arquitetura escolhida para a implementação deste projeto é similar à de outros trabalhos realizados no âmbito de *ray tracing* em GPU já citados, como [Pedrosa 2012].
As informações salvas em texturas pela técnica de *deferred shading* (posição,vetor normal, cor do objeto, entre outros) são necessárias para calcular a iluminação local naquele fragmento. Estes dados também podem ser utilizados para calcular a iluminação na técnica de *ray tracing*, além de serem necessários para o cálculo de sombras, e reflexões.
Assim, ao combinar *ray tracing* com *deferred shading*, utilizamos o processo de rasterização para fundamentalmente eliminar um nível de recursão do *ray tracer*.
Em um *ray tracer* tradicional, para cada pixel é necessário descobrir, percorrendo todos os elementos da cena, qual objeto o raio partindo do ponto de visão do observador até o plano da imagem atinge. Na arquitetura aqui implementada, este passo se torna desnecessário.

3.1 Visão Geral

O projeto foi implementado na linguagem C++, utilizando OpenGL e a biblioteca GLFW. Também foram utilizadas a biblioteca de leitura de imagens stb_image[1], a biblioteca glm[2] para cálculos matemáticos, algumas funções auxiliares[3] e uma classe GLSLShader implementada em [Movania 2013].

Em linhas gerais, o *ray tracer* desenvolvido funciona da seguinte maneira :

- Primeiramente objetos de uma cena são lidos e armazenados em estruturas de dados do programa em C++.
- Dois *shader programs* são criados : o primeiro apenas salva os dados(posição, vetor normal, entre outros) dos objetos em texturas em tela cheia. O segundo é o *ray tracer* propriamente dito.
- Um *framebuffer* é criado para servir como alvo do primeiro passo de renderização. Para este framebuffer são definidos vários *color attachments*. Nestes attachments serão salvas as texturas em tela cheia.
- Uma lista de objetos da cena é percorrida. Cada objeto é desenhado normalmente, utilizando o primeiro *shader program*. O *fragment shader* deste programa salva os atributos de cada fragmento nas texturas do *framebuffer*.
- Após este passo, as texturas em tela cheia são lidas pelo *ray tracer*, que utiliza destes dados, além de buffers contendo os vértices, os materiais e as texturas dos objetos para calcular a iluminação da cena, gerar sombras e reflexões.

Um fluxograma geral da arquitetura é mostrada na figura 1.

[1] https://github.com/nothings/stb
[2] https://glm.g-truc.net
[3] https://github.com/capnramses/antons_opengl_tutorials_book

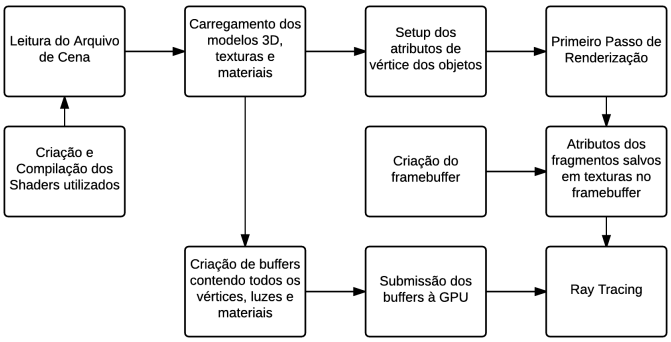


Figura 1: *Fluxograma da arquitetura implementada*

3.2 Carregamento da Cena

Inicialmente, o programa carrega um arquivo descritor de cena, passado como parâmetro. Este arquivo segue um formato simples, criado para esta aplicação. O arquivo contem informação de objetos e luzes. Linhas iniciadas pelo caractere 'o' se referem a um objeto. O nome do objeto vem logo após, seguido de três números separados por espaços, que indicam a posição do objeto na cena, e um quarto número que indica seu índice de reflexão. Linhas iniciadas pelo caractere 'l' se referem a luzes da cena. Os primeiros três valores em uma linha de luz indicam sua posição. Os valores seguintes representam três triplas, que representam os componentes rgb dos parâmetros especular, difuso e ambiente de cada luz. Um exemplo de arquivo é exibido na figura 2.

```
#comentário
#o nomeObjeto posicaoX posicaoY posicaoZ reflectividade
#objeto não reflexivo localizado na origem
o cube 0.0 0.0 0.0 0.0
#objeto parcialmente reflexivo
o sphere 3.0 1.0 2.0 0.5

#luz posicao(xyz) especular(rgb) difuso(rgb) ambiente(rgb)
l 0.0 10.0 2.0 1.0 0.0 0.0 1.0 0.0 0.0 0.1 0.1 0.1
```

Figura 2: *Arquivo de cena*

Cada objeto carregado possui um nome. Este nome identifica o modelo 3D do objeto, carregado de um arquivo obj, um formato comum de representação de modelos 3D, escolhido neste caso pela simplicidade de sua leitura. Além disso, o nome do objeto indica também o arquivo que contém as informações do material de sua superfície, no formato mtl. Do arquivo mtl são lidos os componentes especular, difuso, ambiente e o expoente especular do material do objeto. Por fim, o nome também é utilizado para o carregamento da textura do objeto. Como o *ray tracer* precisa ter acesso às texturas de todos objetos, e em tempo de compilação a quantidade destes é desconhecida, foi decidido pelo uso de um *texture array*. Um *texture array* funciona de maneira similar à uma textura 3D, entretanto o que seria o componente de profundidade desta é, neste caso, utilizado como índice da textura no *array*. Uma desvantagem deste método é que existe a necessidade de que todas as texturas possuam a mesma dimensão.

3.3 Arquitetura da Aplicação C++

A organização das classes do programa está ilustrada no diagrama da figura 3

A classe GLSLShader é utilizada para facilitar a troca dos *shader programs* durante a execução do programa. A classe Scene gerencia os objetos e luzes da cena. Foi utilizado o *design pattern singleton*, existindo apenas uma instancia da classe em toda a aplicação. A classe Object armazena os atributos de cada objetos, como aqueles lidos do arquivo obj, como um vetor a posicao,normal e coordenadas de textura de todos seus vértices. Os objetos da classe Object possuem uma referencia ao *shader* que é utilizado, no primeiro passo de renderização, por aquele objeto. Na implementação atual, todos os objetos utilizam o mesmo *shader*. Outras estruturas de da-

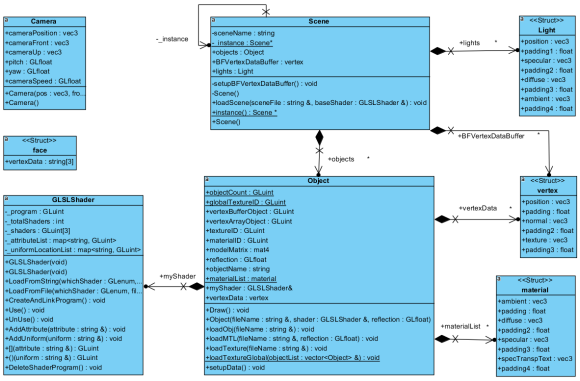


Figura 3: *Diagrama de classes*

dos utilizadas são *structs* para o armazenamento de informações de vértices, materiais, luzes e a câmera da cena.

A classe Scene possui três membros importantes : um vetor de objetos, um vetor de luzes e um vetor contendo todos os vértices da cena, já transformados em sua posição em espaço de mundo. O vetor de objetos é utilizado pelo primeiro passo da renderização para saber quais *vertex buffer* e *vertex array object* serão utilizados (estes são únicos a cada objeto). Também é armazenado em cada objeto sua *model matrix*, que é multiplicada pelos vértices no *vertex shader* do primeiro passo de renderização. Além disso, a classe Object armazena, como membro estático da classe, uma lista de materiais.

Os demais vetores da classe Scene são utilizados pelo *ray tracer* através de um *Shader Storage Buffer Object*, ou SSBO. Este tipo de estrutura de dados, disponível a partir da versão 4.3 do OpenGL, tem como objetivo facilitar o compartilhamento de dados entre a GPU e a CPU. Salvar dados que devem ser utilizados por um *shader* em uma textura, como é feito em [Movania 2013] pode funcionar de maneira aceitável quando os dados são baseados em imagens. Porém em outros casos, grandes blocos de dados estruturados podem ser necessários [Shreiner et al. 2013]. Um vantagem do SSBO é, portanto um mapeamento mais coerente entre as estruturas de dados da aplicação em CPU com as estruturas utilizadas pelos *shader programs*. Além disso, além de lido pelo *shader*, ele pode também ser escrito. O SSBO é criado como outros *buffers* do OpenGL, gerando um identificador para o *buffer*, alocando o espaço necessário. Em seguida o vetor(ou outro *buffer* de memória) da aplicação em CPU é mapeado para o *buffer* do OpenGL. Mais detalhes sobre o uso de SSBOs encontram-se em [Shreiner et al. 2013]. O código na listagem 1 ilustra como este processo é realizado na aplicação descrita. A listagem 2 exibe o código do *buffer* correspondente no *shader* do *ray tracer*.

```
glGenBuffers(1, &posSSbo);
glBindBuffer(GL_SHADER_STORAGE_BUFFER, posSSbo);
glBufferData(GL_SHADER_STORAGE_BUFFER, (sizeof(vertex)) * Scene::instance()->
        BFVertexDataBuffer.size(), NULL, GL_DYNAMIC_COPY);
GLint bufMask = GL_MAP_WRITE_BIT | GL_MAP_INVALIDATE_BUFFER_BIT;
vertex *mapped = reinterpret_cast<vertex *>(glMapBufferRange(
        GL_SHADER_STORAGE_BUFFER, 0, (sizeof(vertex)) * Scene::instance()->
        BFVertexDataBuffer.size(), bufMask));
std::copy(Scene::instance()->BFVertexDataBuffer.begin(), Scene::instance()->
        BFVertexDataBuffer.end(), mapped);
glUnmapBuffer(GL_SHADER_STORAGE_BUFFER);
glBindBufferBase(GL_SHADER_STORAGE_BUFFER, 4, posSSbo);
```

Listing 1: *Criação de um Shader Storage Buffer Object*

```
struct vertex{
  vec3 position;
  vec3 normal;
  vec3 texture;
}

layout (std430, binding = 4)  buffer bufferObject{
  vertex vertices[];
}
```

Listing 2: *Buffer correspondente em um shader*

3.4 Framebuffer e Texturas

A criação do *framebuffer* é feita ao ser iniciada a aplicação. Juntamente são criadas as texturas em tela cheia que serão utilizadas para salvar a posição, vetor normal,textura, o índice global de textura e material do objeto, e o índice de reflexão. Alguns destes dados ocupam inteiramente uma textura com componentes RGBA (como a posição, em coordenadas homogêneas) enquanto outras, como vetor normal, ocupam apenas parte dos componentes RGBA. Assim, dois atributos diferentes podem compartilhar a mesma textura. Por exemplo, o índice de reflexão é armazenado no componente alpha da textura que armazena o vetor normal (que ocupa os componentes RGB). Depois de criadas, estas texturas são alocadas aos *color attachments* do *framebuffer*, e por fim é indicado ao OpenGL que a saída do *framebuffer* deve ser desenhada nestes *color attachments*.

3.5 Loop de Renderização

O código (simplificado) do *loop* de renderização é exibido abaixo, na listagem 3. Os passos do *loop* podem ser discriminados da seguinte forma:

- O *framebuffer* criado é setado como o *framebuffer* alvo.
- A unidade de textura GL_TEXTURE0 é setada como ativa e o *array* de texturas é setado como a textura atual.
- O vetor de objetos da cena é percorrido. Os *uniforms* do *vertex shader* são setados(índice de reflexão, matrizes de projeção, visualização e modelo)
- O comando glDrawArrays é emitido para a GPU, para cada objeto, com o *vertex buffer* específico daquele objeto.
- O *vertex shader* do primeiro estágio recebe como atributos de vértice a posição, o vetor normal, as coordenadas de textura e o índice da textura/material do objeto.
- A posição do vértice é multiplicada pelas matriz de modelo, e passada como variável de saída para o *fragment shader*. O vetor normal as coordenadas de textura e o índice de reflexão também são passados.
- A variável gl_Position, saída padrão do *vertex shader*, recebe a posição multiplicada pelas matrizes de projeção, visualização e modelo, em coordenadas homogêneas.
- O *fragment shader* salva os valores que recebeu de entrada nos buffers das texturas do framebuffer.
- Encerrando o primeiro passo, o framebuffer ativo é trocado pelo padrão, pois agora estaremos desenhando na tela.
- As texturas criadas no primeiro passo são setadas e as unidades de textura do OpenGL ativadas.
- O *shader program* do *ray tracer* é ativado, e um quadrado em tela cheia é desenhado
- O *vertex shader* deste programa apenas transforma as coordenadas dos vértices (que serão interpoladas pelo rasterizador) para espaço de textura, passando estas coordenadas para o *fragment shader*. Estas coordenadas serão utilizadas para recuperar os dados das texturas do primeiro passo. O programa agora está pronto para iniciar o *ray tracing*.

```
glBindFramebuffer(GL_FRAMEBUFFER, frameBuffer);
glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_2D_ARRAY, Object::globalTextureID);
glm::mat4 viewMatrix = glm::lookAt(camera.cameraPosition, camera.
        cameraPosition + camera.cameraFront, camera.cameraUp);

for (auto curObject : Scene::instance()->objects){
  curObject.myShader.Use();
  glBindVertexArray(curObject.vertexArrayObject);
  glUniform1f(curObject.myShader("reflection"), curObject.reflection);
  glUniformMatrix4fv(curObject.myShader("perspectiveMatrix"), 1, GL_FALSE, glm
      ::value_ptr(perspectiveMatrix));
  glUniformMatrix4fv(curObject.myShader("viewMatrix"), 1, GL_FALSE, glm::
      value_ptr(viewMatrix));
  glUniformMatrix4fv(curObject.myShader("modelMatrix"), 1, GL_FALSE, glm::
      value_ptr(curObject.modelMatrix));
  glDrawArrays(GL_TRIANGLES, 0, curObject.vertexData.size());
  curObject.myShader.UnUse();
}

glBindFramebuffer(GL_FRAMEBUFFER, 0);
glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_2D, frameBufferTexture);
glActiveTexture(GL_TEXTURE1);
  glBindTexture(GL_TEXTURE_2D, framBufferNormalTexture);
glActiveTexture(GL_TEXTURE2);
glBindTexture(GL_TEXTURE_2D, frameBufferPositionTexture);
glActiveTexture(GL_TEXTURE3);
glBindTexture(GL_TEXTURE_2D_ARRAY, Object::globalTextureID);
glUseProgram(secondPassShaderProgram);
```

```
GLuint camPosLoc = glGetUniformLocation(secondPassShaderProgram, "cameraPosition");
glUniform3f(camPosLoc, camera.cameraPosition.x, camera.cameraPosition.y, camera.cameraPosition.z);

glBindVertexArray(secondPassVao);
glDrawArrays(GL_TRIANGLES, 0, 6);
```

Listing 3: *Código do loop de renderização*

3.6 Ray Tracing

O *ray tracer* propriamente é executado no *fragment shader* do segundo programa. Ele inicia utilizando as coordenadas do mapeamento de textura em tela cheia para selecionar os texels de cada textura que equivalem ao fragmento atual sendo executado pelo programa. Estes texels contêm os dados necessário para ser realizado o primeiro passo do *ray tracing*. A amostragem dos texels está demonstrada na listagem 4.

```
vec3 position = vec3(texture(positionTexture,textureCoordinate));
vec4 nData = texture(normalTexture,textureCoordinate);
vec3 normal = nData.rgb;
float reflection = nData.a;
vec4 colorData = (texture(colorTexture,textureCoordinate));
vec3 color = colorData.rgb;
float id = colorData.a;
int matId = int(id);
material curMaterial = materials[matId];
```

Listing 4: *Setup do passo inicial do ray tracer*

Os dados lidos das texturas podem ser facilmente visualizados na tela, sendo apenas necessário setar a cor final do *fragment shader* para aquele valor. A figura 4 demonstra uma cena finalizada juntamente com todos os dados lidos das texturas.

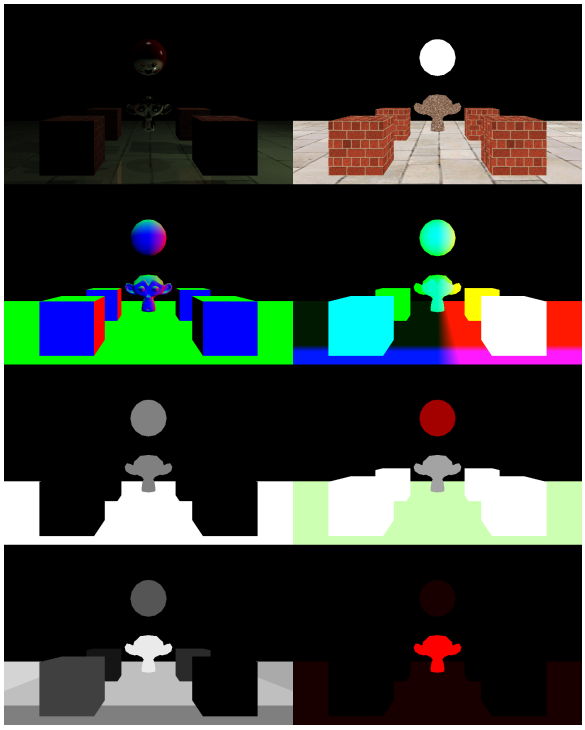


Figura 4: *Imagem Final e dados obtidos de textura. De cima para baixo, e da esquerda para direita : Imagem Final, cor das texturas do fragmento, vetor normal, posição em espaço de mundo, componente especular do material, componente difuso, id do material e índice de reflexão*

Algumas estruturas são criadas no *fragment shader*, para auxiliar na organização do código. Uma destas estruturas é denominada de RayCastHit, que é retornada pela função que verifica a colisão de um raio com os triângulos da cena. Esta estrutura retorna a distancia percorrida pelo raio, as coordenadas baricêntricas do ponto do triângulo que foi atingido, a normal do ponto atingido e o id (no array de vértices) do triângulo.
A estrutura Vertex é utilizada para armazenar os dados da geometria de cena. Ela possui três variáveis de tipo vec3. A posição do vértice em espaço de mundo, o vetor normal do vértice, e as coordenadas de textura do vértice. As coordenadas de textura utilizam apenas dois floats, assim o terceiro valor do vetor é utilizado para armazenar o índice de textura/material do objeto ao qual aquele vértice pertence.
A estrutura material contem os componentes ambiente,especular e difuso do material. Também contem o expoente especular do material e o índice de reflexão do material. Estes dois valores são armazenados juntos em um vec3 de floats.
Uma estrutura contendo três vértices denominada Triangle é utilizada, entretanto os triângulos não estão armazenados diretamente na memória. O vetor que contém a geometria da cena apenas contém os vértices. Estes vértices acabam duplicados, pois não foi utilizado um *index buffer* para decidir quais vértices pertencem a cada triangulo. Assim, o primeiro triangulo é o que contem o vértice de índice zero no vetor de vértices, mais os dois vértices seguintes. O próximo triangulo tem seu início no vértice 4. Este valor é utilizado como índice do triângulo. Além disso, estruturas encapsulando propriedades de um raio e de uma fonte de luz são utilizadas. As luzes são armazenadas em um SSBO, lido do arquivo de cena e passado para o *shader* pela aplicação em CPU. O mesmo ocorre com o vetor de vértices e o vetor de materiais. Outro buffer importante é o *texture array* contendo todas as texturas da cena. Ele é necessário neste estágio para ser possível acessar a textura de um objeto que tenha sido refletido por um objeto visível diretamente pela câmera.

O algoritmo de ray tracing é, em geral implementado de maneira recursiva. Isto não é possível em GLSL. Assim, o algoritmo deve ser adaptado para funcionar de maneira iterativa. A execução do *ray tracer* adaptado pode ser discriminada da seguinte forma:

- Depois de setados os valores passados do passo anterior (que equivaleriam em um *ray tracer* tradicional àqueles retornados pelos raios de câmera), um *loop* é iniciado, tendo limite de execução um valor previamente definido, tendo em vista simular a recursão. O primeiro passo do *loop* é o cálculo da iluminação do fragmento. O cálculo de iluminação percorre todas as luzes da cena, e procura algum objeto entre o fragmento e a luz. Caso exista, aquele ponto está em sombra(desta luz específica).

- A verificação da existência de um objeto entre a luz e o fragmento é realizada percorrendo todo o vetor de vértices, agregando os vértices de três em três e utilizando o algoritmo de intersecção de raio-triangulo de Möller-Trumbore [Möller and Trumbore 1997]. O algoritmo retorna (caso o raio acerte um triângulo), além da posição do acerto, as coordenadas baricêntricas do ponto no triângulo. Estas coordenadas são utilizadas posteriormente para interpolar o vetor normal e as coordenadas de textura das reflexões (as reflexões tem acesso apenas aos dados dos vértices, nao aos dados em espaço de tela. Caso a interpolação não fosse feita, o vetor normal seria o mesmo para todos os pontos do triângulo, e as texturas não funcionariam).

- Caso não esteja em sombra, a iluminação local é calculada. Esta iluminação é calculada utilizando o modelo de phong [Phong 1975]. Este modelo foi escolhido devido à sua familiaridade e simplicidade, tendo em vista o objetivo de se obter um desempenho de renderização que se aproxime de tempo real. Os componentes do material do objeto e da luz atual são utilizados para o cálculo da iluminação, permitindo assim luzes coloridas, por exemplo, e materiais que simulem superfícies distintas, como metal e plástico. A textura do fragmento também é utilizada na iluminação.

- Depois de simular os raios primários e calcular as sombras, o programa verifica se o fragmento é reflexivo. Se não for, a cor de saída do fragmento é setada e a execução termina. Caso for reflexivo, um raio de reflexão é setado, com origem na posição no espaço de mundo do fragmento, e direção na reflexão do vetor de direção do observador ao ponto do fragmento, em torno do vetor normal do ponto, . O raio de reflexão percorre toda a geometria da cena. Caso não encontre um objeto, a cor final do fragmento é setada e ajustada de acordo com o índice de reflexão do objeto. A execução termina

- Caso for encontrado um objeto, os dados recebidos pela estrutura RayCastHit passada à função que percorre a geometria da cena são utilizados para setar os valores necessários ao cálculo de iluminação do ponto de reflexão. Caso o *loop* não tenha atingido o limite de execução, voltamos ao passo inicial de calcular a iluminação local. Caso o limite de execução tenha sido atingido, a execução termina e a cor de saída do fragmento é setada.

O código do *loop* do *ray tracer* é mostrado na listagem 5

```
RayCastHit hit;
hit.p = position;
hit.normal = normalize(normal);
vec3 textureSample = vec3(texture(colorTexture,textureCoordinate));
int depth = 5;
vec3 eyePos = cameraPosition;

for(int i = 0; i < depth; i++){
    vec3 localColor = vec3(0.0);
  for(int i = 0;i < lightsArray.length(); i++){
      Light newTestLigth = lightsArray[i];
    localColor += shade(cameraPosition,newTestLigth,hit,curMaterial,
        textureSample);
  }

  finalColor += localColor*(1.0-reflection)*frac;

  if(reflection < 0.01)
      break;

    vec3 ReflectRayOrigin = hit.p;
  vec3 camRay = normalize(ReflectRayOrigin - eyePos);
  vec3 ReflectRayDirection = reflect(camRay,hit.normal);
  Ray reflectionRay = Ray(ReflectRayOrigin,ReflectRayDirection);

  if(!worldHit(reflectionRay,0.01,infinity,hit,false))//{
      break;

  int id = hit.objectID;
  matId = int(vertices[id].texture.p);
  curMaterial = materials[matId];
  textureSample = getTextureValue(hit.u,hit.v,id);
  eyePos = ReflectRayOrigin;
  reflection = curMaterial.specTranspText.y;
}
```

Listing 5: *Ray Tracing*

4 Resultados

A qualidade visual inerente à técnica de ray tracing é percebida nas imagens geradas pelo programa implementado. As sombras tem boa definição, e a implementação de sombras de múltiplas fontes de luz é trivial. As reflexões também apresentam boa qualidade, refletindo todos os elementos da cena, corretamente, até o nível máximo de profundidade definido.
Em geral em aplicações gráficas em tempo real, estes efeitos são implementados através de técnicas que apenas aproximam os resultados ideais, muitas vezes com baixa qualidade. Exemplos são mapeamento de textura de baixa resolução, que apresenta serrilhados, e reflexões baseadas em *enviroment mapping*, onde apenas um *skybox* é refletido, gerando uma reflexão incompleta, pois os demais elementos da cena são ignorados.

4.1 Desempenho

O desempenho do sistema varia de acordo com a complexidade da cena. Uma cena com cerca de 8060 triângulos, nenhum objeto reflexivo e quatro fontes de luz, teve tempo médio de renderização de 1964.97 milissegundos por frame, em uma GPU Radeon R7 260x. Uma cena com 1252 triângulos, oito objetos reflexivos e duas luzes teve tempo médio de renderização de 97.62 milissegundos. Estes valores equivalem a cerca de 0.5 e 10 frames por segundo, respectivamente. As cenas criadas são exibidas nas figuras 5 e 6.

Fica claro, ao analisar os resultados, que a quantidade de objetos, a quantidade de luzes e a reflexividade afetam fortemente o desempenho do *ray tracer*. Ao se remover a reflexão dos objetos da primeira cena de testes, o fps aumenta 100%. Entretanto este tipo de adaptação não é realista como forma de otimização, pois elimina uma das principais razões para o uso de *ray tracing* em primeiro lugar. Outras técnicas de otimização seriam indicadas para melhorar o desempenho até ser atingido um tempo de renderização aceitável para um aplicação em tempo real.

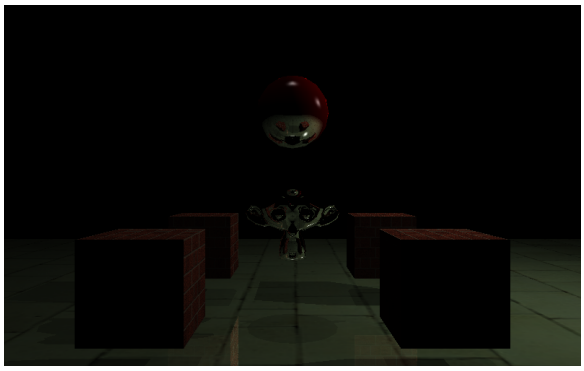


Figura 5: *Primeira cena de testes*

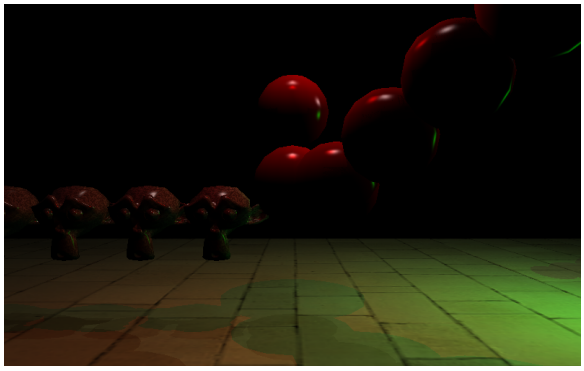


Figura 6: *Segunda cena de testes*

4.2 Comparação de arquiteturas

Para testar o possível ganho de performance decorrente do uso de *deferred shading*, um teste foi realizado. As duas cenas descritas acima foram utilizadas. Uma versão do *ray tracer* sem *deferred shading* foi criada a partir do projeto original. Esta versão ignora o primeiro passo de renderização, gerando raios de câmera a partir dos dados disponíveis no *shader* do *ray tracer*. As cenas foram rodadas pelos dois projetos, e seu desempenho comparado. Os resultados se encontram na tabela 1.

Tabela 1: *Resultados*

	Cena 1, deferred	**Cena2, deferred**	**Cena 1, s/ deferred**	**Cena 2, s/deferred**
ms/frame	97.62	1964.97	178.91	2582.55
FPS	10.24	0.5089	5.58	0.3872
	ms/frame	**FPS**		
Var. Cena1	83.28%	45.43%		
Var.Cena2	31.42%	23.91%		

4.3 Análise dos resultados

Fica evidente a vantagem que a arquitetura utilizando *deferred shading* possui. Sua performance foi melhor em ambas as cenas, na primeira obtendo um resultado quase duas vezes mais rápido. Comparando as peculiaridades das cenas, pode-se supor que a vantagem da arquitetura híbrida tende a diminuir à medida que a quantidade de triângulos e luzes na cena aumenta. Isso faz sentido, pois o cálculo de sombras sempre percorre o vetor de vértices. A vantagem do *deferred shading* decorre da não necessidade da geração dos raios primários (os gerados da posição do observador para a cena). Com o resultado dos raios primários, são calculados os raios secundários - sombra e reflexão. Assim quanto maior a quantidade de luzes e vértices, mais a vantagem da arquitetura híbrida diminui.

5 Conclusões e Trabalhos Futuros

Este trabalho apresentou uma arquitetura híbrida que implementa a técnica de *ray tracing* em GPU utilizando *deferred shading* como forma de otimização. O desempenho do projeto não chega a atingir uma velocidade de renderização adequada à aplicações em tempo real, exceto em casos de cenas muito simples. Porém, a utilização da técnica de *deferred shading* permitiu uma boa melhorar no desempenho do projeto. A principal razão do desempenho insatisfatório decorre da não implementação de estruturas de dados espaciais, utilizadas para diminuir a quantidade de cálculos de interseção entre raios e triângulos. Existe uma grande variedade destas estruturas, tais como Octrees, kd-trees e *Bounding Volume Hierarchy*. Cada uma destas estruturas possui vantagens e desvantagens, e um estudo aprofundado delas foge do escopo deste projeto. Uma consideração a ser feita é que este *ray tracer* foi implementado em GPU, e certas técnicas usuais na programação em CPU não são possíveis em GPU, como o uso de recursão e ponteiros. Assim, para ser utilizada, uma estrutura de dados espacial deverá ser adaptada ao contexto da aplicação. Um modelo de Octree em GPU é descrito em [Lefebvre et al. 2005], onde elementos de uma textura são utilizados como forma de acesso indireto a dados armazenados na própria textura. Outra otimização possível, é a utilização de uma heurística como forma de selecionar apenas alguns objetos da cena para serem renderizados pelo *ray tracer*, deixando os demais a serem renderizados tradicionalmente, como é feito em [Pedrosa 2012].

Referências

BIKKER, J., AND VAN SCHIJNDEL, J. 2013. The brigade renderer: A path tracer for real-time games. *International Journal of Computer Games Technology 2013*.

CABELEIRA, J. P. G. 2010. *Combining rasterization and ray tracing techniques to approximate global illumination in real-time*. PhD thesis.

COOK, S. 2013. *CUDA Programming*. Elsevier.

GARCÍA, A., ÁVILA, F., MURGUÍA, S., AND REYES, L. 2012. Interactive Ray Tracing Using the Compute Shader in DirectX11. In *GPU Pro 3*, W. Engel, Ed. A K Peters/CRC Press, ch. 4-3, 353–376.

LEFEBVRE, S., HORNUS, S., NEYRET, F., ET AL. 2005. Octree textures on the gpu. *GPU gems 2*, 595–613.

MÖLLER, T., AND TRUMBORE, B. 1997. Fast, minimum storage ray-triangle intersection. *J. Graph. Tools 2*, 1 (Oct.), 21–28.

MOVANIA, M. 2013. *OpenGL Development Cookbook*. Quick answers to common problems. Packt Publishing.

PARKER, S. G., BIGLER, J., DIETRICH, A., FRIEDRICH, H., HOBEROCK, J., LUEBKE, D., MCALLISTER, D., MCGUIRE, M., MORLEY, K., ROBISON, A., AND STICH, M. 2010. Optix: A general purpose ray tracing engine. *ACM Trans. Graph. 29*, 4 (July), 66:1–66:13.

PEDROSA, F. ; SABINO, T. L. R. . M. A. A. . P. P. . C. E. W. G. 2012. An adaptive hybrid raster ray-tracing renderer to real-time applications. In *XI Simpósio Brasileiro de Jogos e Entretenimento Digital (SBGames 2012), 2012, Brasilia. SBGames 2011 - Computing Track. Porto Alegre: Sociedade Brasileira da Computação, 2012. v. 11.*, IEEE, 101–107.

PHONG, B. T. 1975. Illumination for computer generated pictures. *Commun. ACM 18*, 6 (June), 311–317.

SABINO, T. L. R., ANDRADE, P., LATTARI, L. G., CLUA, E., MONTENEGRO, A., AND PAGLIOSA, P. A. 2011. Efficient use of in-game ray-tracing techniques. *SBC—Proceedings of SBGAMES*.

SHREINER, D., SELLERS, G., KESSENICH, J. M., AND LICEA-KANE, B. M. 2013. *OpenGL Programming Guide: The Official Guide to Learning OpenGL, Version 4.3*, 8th ed. Addison-Wesley Professional.

WANG, Y., LIU, C., AND DENG, Y. 2014. A feasibility study of ray tracing on mobile gpus. In *SIGGRAPH Asia 2014 Mobile Graphics and Interactive Applications*, ACM, 3.